

```
==> ./models.py <==  
"""
```

```
Database models for Historical Stock Market Simulator  
SQLite database for persistent storage  
"""
```

```
import sqlite3  
import json  
from datetime import datetime  
from typing import Dict, Optional, List  
import os  
import time
```

```
DATABASE_FILE = 'stock_simulator.db'  
DEFAULT_START_DATE = '1928-01-01'
```

```
def get_db_connection():  
    """Get a database connection"""  
    conn = sqlite3.connect(DATABASE_FILE, timeout=10)  
    conn.row_factory = sqlite3.Row  
    conn.execute('PRAGMA journal_mode=WAL')  
    conn.execute('PRAGMA busy_timeout = 10000')  
    return conn
```

```
def init_db():  
    """Initialize the database with required tables"""  
    conn = get_db_connection()  
    cursor = conn.cursor()  
  
    # Create rooms table  
    cursor.execute('''  
        CREATE TABLE IF NOT EXISTS rooms (  
            code TEXT PRIMARY KEY,  
            start_date TEXT NOT NULL,  
            current_date TEXT NOT NULL,  
            game_state TEXT DEFAULT 'paused',  
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
        )  
    ''')  
  
    # Create players table  
    cursor.execute('''  
        CREATE TABLE IF NOT EXISTS players (  
            id INTEGER PRIMARY KEY AUTOINCREMENT,  
            room_code TEXT NOT NULL,  
            username TEXT NOT NULL,  
            cash REAL DEFAULT 100.0,  
            holdings TEXT DEFAULT '{}',  
            is_admin BOOLEAN DEFAULT FALSE,  
            FOREIGN KEY (room_code) REFERENCES rooms(code),  
            UNIQUE(room_code, username)  
        )  
    ''')  
  
    conn.commit()  
    conn.close()  
    print("Database initialized successfully")
```

```
class Room:
```

```
"""Room model for managing game rooms"""
```

```
def __init__(self, code: str, start_date: str):  
    self.code = code  
    self.start_date = start_date  
    self.current_date = start_date  
    self.game_state = 'paused'
```

```
@staticmethod
```

```
def create(code: str, start_date: str) -> 'Room':
```

```
    """Create a new room"""
```

```
    start_date = start_date or DEFAULT_START_DATE
```

```
    current_date = DEFAULT_START_DATE
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute('''
```

```
        INSERT INTO rooms (code, start_date, current_date, game_state)
```

```
        VALUES (?, ?, ?, 'paused')
```

```
    ''', (code, start_date, current_date))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    room = Room(code, start_date)
```

```
    room.current_date = current_date
```

```
    return room
```

```
@staticmethod
```

```
def get(code: str) -> Optional['Room']:
```

```
    """Get a room by code"""
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute('SELECT * FROM rooms WHERE code = ?', (code,))
```

```
    row = cursor.fetchone()
```

```
    conn.close()
```

```
    if row:
```

```
        room = Room(row['code'], row['start_date'])
```

```
        room.current_date = row['current_date']
```

```
        room.game_state = row['game_state']
```

```
        return room
```

```
    return None
```

```
@staticmethod
```

```
def exists(code: str) -> bool:
```

```
    """Check if a room exists"""
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute('SELECT 1 FROM rooms WHERE code = ?', (code,))
```

```
    result = cursor.fetchone()
```

```
    conn.close()
```

```
    return result is not None
```

```
def save(self):
```

```
    """Save room state to database"""
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute('''
```

```
        UPDATE rooms
```

```

        SET current_date = ?, game_state = ?
        WHERE code = ?
    ''', (self.current_date, self.game_state, self.code))

    conn.commit()
    conn.close()

```

```

def update_date(self, new_date: str):
    """Update the current game date"""
    self.current_date = new_date
    self.save()

def set_state(self, state: str):
    """Set the game state (paused/playing)"""
    self.game_state = state
    self.save()

def reset(self):
    """Reset the room to initial state"""
    self.current_date = self.start_date
    self.game_state = 'paused'
    self.save()

```

```

class Player:
    """Player model for managing students"""

    def __init__(self, room_code: str, username: str, cash: float = 100.0,
                  holdings: Dict[str, float] = None, is_admin: bool = False):
        self.room_code = room_code
        self.username = username
        self.cash = cash
        self.holdings = holdings or {}
        self.is_admin = is_admin

    @staticmethod
    def create(room_code: str, username: str, is_admin: bool = False) -> 'Player':
        """Create a new player"""
        max_retries = 5
        base_username = username

        for attempt in range(max_retries):
            conn = None
            try:
                conn = get_db_connection()
                cursor = conn.cursor()
                cursor.execute('BEGIN IMMEDIATE')

                candidate = base_username
                counter = 1

                # Find a unique username while holding a write lock to avoid races.
                while True:
                    cursor.execute('''
                        SELECT 1 FROM players
                        WHERE room_code = ? AND username = ?
                    ''', (room_code, candidate))
                    if not cursor.fetchone():
                        break
                    candidate = f"{base_username}{counter}"

```

```

        counter += 1
        cursor.execute('''
            INSERT INTO players (room_code, username, cash, holdings, is
_admin)
            VALUES (?, ?, 100.0, '{}', ?)
        ''', (room_code, candidate, is_admin))
        conn.commit()
        return Player(room_code, candidate, 100.0, {}, is_admin)
    except sqlite3.OperationalError as e:
        if conn:
            conn.rollback()
            if 'locked' in str(e).lower() and attempt < max_retries - 1:
                time.sleep(0.1 * (attempt + 1))
                continue
            raise
        finally:
            if conn:
                conn.close()

    raise RuntimeError('Unable to create player after retries')

@staticmethod
def get(room_code: str, username: str) -> Optional['Player']:
    """Get a player by room and username"""
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute('''
        SELECT * FROM players
        WHERE room_code = ? AND username = ?
    ''', (room_code, username))

    row = cursor.fetchone()
    conn.close()

    if row:
        player = Player(
            row['room_code'],
            row['username'],
            row['cash'],
            json.loads(row['holdings']),
            row['is_admin']
        )
        player.id = row['id']
        return player
    return None

@staticmethod
def get_all_in_room(room_code: str) -> List['Player']:
    """Get all players in a room"""
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute('''
        SELECT * FROM players WHERE room_code = ?
    ''', (room_code,))

    rows = cursor.fetchall()
    conn.close()

    players = []

```

```

    for row in rows:
        player = Player(
            row['room_code'],
            row['username'],
            row['cash'],
            json.loads(row['holdings']),
            row['is_admin']
        )
        player.id = row['id']
        players.append(player)

    return players

def save(self):
    """Save player state to database"""
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute('''
        UPDATE players
        SET cash = ?, holdings = ?
        WHERE room_code = ? AND username = ?
    ''', (self.cash, json.dumps(self.holdings), self.room_code, self.username))

    conn.commit()
    conn.close()

def buy(self, ticker: str, shares: float, price: float) -> bool:
    """Buy shares of a stock"""
    cost = shares * price
    if cost > self.cash:
        return False

    self.cash -= cost
    self.holdings[ticker] = self.holdings.get(ticker, 0) + shares
    self.save()
    return True

def sell(self, ticker: str, shares: float, price: float) -> bool:
    """Sell shares of a stock"""
    current_shares = self.holdings.get(ticker, 0)
    if shares > current_shares:
        return False

    self.cash += shares * price
    self.holdings[ticker] = current_shares - shares
    if self.holdings[ticker] <= 0:
        del self.holdings[ticker]
    self.save()
    return True

def get_net_worth(self, prices: Dict[str, float]) -> float:
    """Calculate total net worth"""
    holdings_value = 0
    for ticker, shares in self.holdings.items():
        if ticker in prices:
            holdings_value += shares * prices[ticker]
    return self.cash + holdings_value

def reset(self):
    """Reset player to initial state"""

```

```
self.cash = 100.0
self.holdings = {}
self.save()
```

```
def to_dict(self, prices: Dict[str, float] = None) -> dict:
    """Convert player to dictionary"""
    net_worth = self.get_net_worth(prices) if prices else self.cash
    return {
        'username': self.username,
        'cash': self.cash,
        'holdings': self.holdings,
        'net_worth': net_worth,
        'is_admin': self.is_admin
    }
```

```
# Initialize database on module import
init_db()
```

```
==> ./data_engine.py <==
```

```
"""
Data Engine for Historical Stock Market Simulator
Fetches and manages historical stock data using yfinance
With fallback to sample data if yfinance fails
"""
```

```
import yfinance as yf
from datetime import datetime, timedelta
from typing import Dict, Optional
import json
import os
import time
import pandas as pd
```

```
# Stock tickers to fetch
TICKERS = ['^DJI', '^GSPC']
```

```
# Company names for display
COMPANY_NAMES = {
    '^DJI': 'Dow Jones Industrial Average',
    '^GSPC': 'S&P 500'
}
```

```
# IPO dates for each ticker
IPO_DATES = {
    '^DJI': '1920-01-01',
    '^GSPC': '1957-03-01'
}
```

```
# Cache file for storing fetched data
CACHE_FILE = 'stock_data_cache.json'
```

```
def generate_sample_data():
    """Generate sample historical data for demonstration"""
    print("Generating sample data...")

    base_date = datetime(1920, 1, 1)
    end_date = datetime(2024, 12, 31)

    # Fallback prices for Jan 1, 1928 (historical reference)
    FALLBACK_PRICES_1928 = {
        '^DJI': 200.00,
```

```

    '^GSPC': 17.50
}

# Sample price data (simplified historical values)
initial_prices = {
    '^DJI': 100.0,
    '^GSPC': 10.0
}

ipo_dates = IPO_DATES.copy()

sample_data = {}

for ticker in TICKERS:
    prices = {}
    ipo = datetime.strptime(ipo_dates[ticker], '%Y-%m-%d')
    price = initial_prices[ticker]
    current_date = base_date

    while current_date <= end_date:
        if current_date >= ipo:
            date_str = current_date.strftime('%Y-%m-%d')

            # Use fallback price for Jan 1, 1928
            if date_str == '1928-01-01':
                price = FALLBACK_PRICES_1928[ticker]
                prices[date_str] = round(price, 2)
            elif current_date.weekday() < 5:
                # Skip weekends for stocks
                # Add realistic variation
                change = (hash(date_str) % 100 - 45) / 1000

                price = price * (1 + change)
                if price < 0.01:
                    price = 0.01
                prices[date_str] = round(price, 2)

            current_date += timedelta(days=1)

        sample_data[ticker] = {
            'prices': prices,
            'ipo_date': ipo_dates[ticker]
        }
    print(f" Generated {len(prices)} days for {ticker}")

return sample_data

```

```

class DataEngine:
    """Manages historical stock data fetching and retrieval"""

    def __init__(self):
        self.stock_data: Dict[str, Dict[str, float]] = {}
        self.ipo_dates: Dict[str, str] = {}
        self._load_or_fetch_data()

    def _load_or_fetch_data(self):
        """Load cached data or fetch from yfinance"""
        if os.path.exists(CACHE_FILE):
            print("Loading cached stock data...")
            self._load_cache()
        return

```

```

print("Attempting to fetch stock data from yfinance...")
self._fetch_all_data()

# If no data was fetched, generate sample data
if not self.stock_data:
    print("yfinance unavailable, generating sample data...")
    self._generate_sample_data()

self._save_cache()

def _load_cache(self):
    """Load data from cache file"""
    try:
        with open(CACHE_FILE, 'r') as f:
            data = json.load(f)
            self.stock_data = data.get('stock_data', {})
            self.ipo_dates = data.get('ipo_dates', {})
        print(f"Loaded data for {len(self.stock_data)} tickers")
    except Exception as e:
        print(f"Error loading cache: {e}")
        self.stock_data = {}
        self.ipo_dates = {}

def _save_cache(self):
    """Save data to cache file"""
    try:
        data = {
            'stock_data': self.stock_data,
            'ipo_dates': self.ipo_dates
        }
        with open(CACHE_FILE, 'w') as f:
            json.dump(data, f)
        print("Stock data cached successfully")
    except Exception as e:
        print(f"Error saving cache: {e}")

def _fetch_all_data(self):
    """Fetch historical data for all tickers"""
    for ticker in TICKERS:
        self._fetch_ticker_data(ticker)
        time.sleep(0.5)

def _fetch_ticker_data(self, ticker: str):
    """Fetch historical data for a single ticker"""
    try:
        print(f"Fetching {ticker}...")
        stock = yf.Ticker(ticker)
        hist = stock.history(period="max")

        if hist.empty:
            hist = stock.history(start="1920-01-01", end="2024-12-31")

        if hist.empty:
            print(f" No data available for {ticker}")
            return

        prices = {}
        for date, row in hist.iterrows():
            if pd.isna(row['Close']):
                date_str = date.strftime('%Y-%m-%d')
                prices[date_str] = float(row['Close'])
    
```



```

        if prices:
            self.stock_data[ticker] = prices
            ipo_date = min(prices.keys())
            self.ipo_dates[ticker] = ipo_date
            print(f"    {ticker}: {len(prices)} days, IPO: {ipo_date}")

    except Exception as e:
        print(f"    Error fetching {ticker}: {e}")

def _generate_sample_data(self):
    """Generate sample data when yfinance fails"""
    sample = generate_sample_data()

    for ticker, data in sample.items():
        self.stock_data[ticker] = data['prices']
        self.ipo_dates[ticker] = data['ipo_date']

def get_price(self, ticker: str, date: str) -> Optional[float]:
    """Get the stock price for a given ticker on a specific date."""
    if ticker not in self.stock_data:
        return None

    ipo_date = self.ipo_dates.get(ticker)
    if ipo_date and date < ipo_date:
        return None

    if date in self.stock_data[ticker]:
        return self.stock_data[ticker][date]

    return self._find_closest_price(ticker, date)

def _find_closest_price(self, ticker: str, target_date: str) -> Optional[float]:
    """Find the closest trading day price before or on the target date"""
    prices = self.stock_data.get(ticker, {})
    if not prices:
        return None

    target = datetime.strptime(target_date, '%Y-%m-%d')

    valid_dates = []
    for date_str in prices.keys():
        date = datetime.strptime(date_str, '%Y-%m-%d')
        if date <= target:
            valid_dates.append(date)

    if not valid_dates:
        return None

    closest = max(valid_dates)
    return prices[closest.strftime('%Y-%m-%d')]

def is_available(self, ticker: str, date: str) -> bool:
    """Check if a stock is available for trading on a given date"""
    ipo_date = self.ipo_dates.get(ticker)
    if not ipo_date:
        return False
    return date >= ipo_date

def get_ipo_date(self, ticker: str) -> Optional[str]:
    """Get the IPO date for a ticker"""

```

```

        return self.ipo_dates.get(ticker)

def get_all_prices(self, date: str) -> Dict[str, float]:
    """Get all stock prices for a given date"""
    prices = {}
    for ticker in TICKERS:
        price = self.get_price(ticker, date)
        if price is not None:
            prices[ticker] = price
    return prices

def get_company_name(self, ticker: str) -> str:
    """Get the company name for a ticker"""
    return COMPANY_NAMES.get(ticker, ticker)

def get_tickers(self) -> list:
    """Get list of all tickers"""
    return TICKERS.copy()

def get_available_tickers(self, date: str) -> list:
    """Get list of tickers available for trading on a given date"""
    available = []
    for ticker in TICKERS:
        if self.is_available(ticker, date):
            available.append(ticker)
    return available

def get_date_range(self) -> tuple:
    """Get the earliest and latest dates available"""
    baseline_start = '1920-01-01'
    all_dates = set()
    for prices in self.stock_data.values():
        all_dates.update(prices.keys())

    if not all_dates:
        return (baseline_start, '2024-12-31')

    return (baseline_start, max(all_dates))

# Global data engine instance
data_engine = DataEngine()
==> ./headlines.py <==
"""
Local historical headlines for the simulator.
"""

HEADLINES = [
    {
        'year': 1914,
        'headline': 'World War I begins after tensions in Europe erupt into full
-scale conflict.',
        'source': 'Historical Archive',
        'url': 'https://en.wikipedia.org/wiki/World_War_I'
    },
    {
        'year': 1918,
        'headline': 'World War I ends as the armistice is signed on November 11.',
        'source': 'Historical Archive',
        'url': 'https://en.wikipedia.org/wiki/Armistice_of_11_November_1918'
    },

```

```

{
  'year': 1929,
  'headline': 'Wall Street crashes, triggering the Great Depression.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Wall_Street_Crash_of_1929'
},
{
  'year': 1939,
  'headline': 'World War II begins with the invasion of Poland.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Invasion_of_Poland'
},
{
  'year': 1945,
  'headline': 'World War II ends after Allied victory in Europe and the Pa
cific.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/End_of_World_War_II'
},
{
  'year': 1957,
  'headline': 'Soviet Union launches Sputnik, opening the Space Age.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Sputnik_1'
},
{
  'year': 1963,
  'headline': 'President John F. Kennedy is assassinated in Dallas.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Assassination_of_John_F._Kennedy'
},
{
  'year': 1969,
  'headline': 'Apollo 11 lands on the Moon; humans walk on the lunar surfa
ce.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Apollo_11'
},
{
  'year': 1973,
  'headline': 'Oil embargo shocks global markets and fuels inflation.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/1973_oil_crisis'
},
{
  'year': 1986,
  'headline': 'Chernobyl nuclear disaster raises worldwide safety concerns
.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Chernobyl_disaster'
},
{
  'year': 1989,
  'headline': 'The Berlin Wall falls, signaling the end of the Cold War er
a.',
  'source': 'Historical Archive',
  'url': 'https://en.wikipedia.org/wiki/Fall_of_the_Berlin_Wall'
},
{
  'year': 1991,
  'headline': 'The Soviet Union dissolves, reshaping global geopolitics.',
  'source': 'Historical Archive',

```

```

        'url': 'https://en.wikipedia.org/wiki/Dissolution_of_the_Soviet_Union'
    },
    'year': 2001,
    'headline': 'September 11 attacks profoundly change global security poli
cy.',
    'source': 'Historical Archive',
    'url': 'https://en.wikipedia.org/wiki/September_11_attacks'
},
    'year': 2008,
    'headline': 'Global financial crisis deepens after major banking failure
s.',
    'source': 'Historical Archive',
    'url': 'https://en.wikipedia.org/wiki/Financial_crisis_of_2007%E2%80%932
008'
},
    'year': 2020,
    'headline': 'COVID-19 pandemic disrupts economies and daily life worldwi
de.',
    'source': 'Historical Archive',
    'url': 'https://en.wikipedia.org/wiki/COVID-19_pandemic'
}
]

```

```

def get_headline_for_year(year: int):
    """Return the configured headline for an exact year match."""
    for item in HEADLINES:
        if item['year'] == year:
            return item
    return None

```

```

==> ./game_clock.py <==
"""

```

```

Game Clock for Historical Stock Market Simulator
Manages time progression and broadcasting
"""

```

```

import threading
import time
import calendar
from datetime import datetime, timedelta
from typing import Dict, Callable, Optional
from data_engine import data_engine

```

```

class GameClock:
    """Manages the game clock and time progression"""

    # Presentation speed: every second advances about 3 months.
    MONTHS_PER_TICK = 3
    SECONDS_PER_TICK = 1.0

    def __init__(self, room_code: str):
        self.room_code = room_code
        self.current_date: str = '1920-01-01'
        self.game_state: str = 'paused'
        self._timer: Optional[threading.Thread] = None
        self._running: bool = False
        self._broadcast_callback: Optional[Callable] = None

```

```

self._sp500_notified: bool = False

def set_broadcast_callback(self, callback: Callable):
    """Set the callback function for broadcasting updates"""
    self._broadcast_callback = callback

def start(self):
    """Start the game clock"""
    if self._running:
        return

    self._running = True
    self._timer = threading.Thread(target=self._run_clock, daemon=True)
    self._timer.start()

def stop(self):
    """Stop the game clock"""
    self._running = False
    if self._timer:
        self._timer.join(timeout=2)

def _run_clock(self):
    """Main clock loop - runs in separate thread"""
    while self._running:
        try:
            if self.game_state == 'playing':
                self._advance_time()

                # Check for S&P 500 launch notification
                if not self._sp500_notified and self.current_date >= '1957-0
3-01':
                    self._sp500_notified = True
                    # Import socketio here to avoid circular import
                    from app import socketio
                    socketio.emit('market_event', {
                        'type': 'sp500_launch',
                        'message': 'S&P 500 Launched!',
                        'date': self.current_date
                    }, room=self.room_code)

                    # Broadcast game state updates for the room
                    if self._broadcast_callback:
                        self._broadcast_callback(self.room_code)
        except Exception as e:
            # Keep clock thread alive if a broadcast or emit fails transient
            ly.
            print(f"Clock loop error in room {self.room_code}: {e}")

            time.sleep(self.SECONDS_PER_TICK)

def _advance_time(self):
    """Advance time by approximately three calendar months."""
    current = datetime.strptime(self.current_date, '%Y-%m-%d')

    # Move ahead by fixed month increments (presentation compression mode).
    month_index = (current.month - 1) + self.MONTHS_PER_TICK
    year = current.year + (month_index // 12)
    month = (month_index % 12) + 1
    day = min(current.day, calendar.monthrange(year, month)[1])
    next_day = current.replace(year=year, month=month, day=day)

    # Skip weekends (Saturday=5, Sunday=6)

```

```

while next_day.weekday() >= 5:
    next_day += timedelta(days=1)

# Check if we have data for this date
max_date = data_engine.get_date_range()[1]
next_date_str = next_day.strftime('%Y-%m-%d')

if next_date_str > max_date:
    # Reached end of available data, pause
    self.game_state = 'paused'
    return

```

```

self.current_date = next_date_str

```

```

def set_date(self, date: str):
    """Set the current game date"""
    # Validate date is within bounds
    min_date, max_date = data_engine.get_date_range()

    if date < min_date:
        date = min_date
    elif date > max_date:
        date = max_date

```

```

self.current_date = date

```

```

def set_state(self, state: str):
    """Set the game state (paused/playing)"""
    self.game_state = state
    if state == 'playing':
        self.awaiting_resume = False

    if state == 'playing' and not self._running:
        self.start()
    elif state == 'paused' and self._running:
        self.stop()

```

```

def get_state(self) -> Dict:
    """Get current clock state"""
    prices = data_engine.get_all_prices(self.current_date)
    return {
        'current_date': self.current_date,
        'game_state': self.game_state,
        'prices': prices
    }

```

```

def reset(self, start_date: str):
    """Reset the clock to start date"""
    self.stop()
    self.current_date = start_date
    self.game_state = 'paused'
    self._running = False

```

Global clock manager

```

class ClockManager:
    """Manages multiple game clocks (one per room)"""

```

```

    def __init__(self):
        self._clocks: Dict[str, GameClock] = {}

```

```

def get_clock(self, room_code: str) -> GameClock:
    """Get or create a clock for a room"""
    if room_code not in self._clocks:
        self._clocks[room_code] = GameClock(room_code)
    return self._clocks[room_code]

def remove_clock(self, room_code: str):
    """Remove a clock when room is deleted"""
    if room_code in self._clocks:
        self._clocks[room_code].stop()
        del self._clocks[room_code]

# Global clock manager instance
clock_manager = ClockManager()
==> ./app.py <==
"""

Main Flask Application for Historical Stock Market Simulator
"""

import os
import random
import string
from flask import Flask, render_template, request, jsonify, session, redirect, u
rl_for
from flask_socketio import SocketIO, emit, join_room, leave_room

from models import Room, Player, init_db
from data_engine import data_engine, TICKERS, COMPANY_NAMES
from game_clock import clock_manager
from headlines_data import HEADLINES_BY_YEAR

app = Flask(__name__, template_folder='templates')
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'stock-simulator-secret-
key')
DEFAULT_START_DATE = '1928-01-01'

# Initialize SocketIO with threading for development
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')

def generate_room_code(length: int = 5) -> str:
    """Generate a random alphanumeric room code"""
    chars = string.ascii_uppercase + string.digits
    return ''.join(random.choice(chars) for _ in range(length))

def broadcast_leaderboard(room_code: str):
    """Broadcast updated leaderboard to all players in a room"""
    players = Player.get_all_in_room(room_code)
    clock = clock_manager.get_clock(room_code)
    prices = data_engine.get_all_prices(clock.current_date)

    # Calculate net worth for each player
    leaderboard = []
    for player in players:
        leaderboard.append(player.to_dict(prices))

    # Sort by net worth (descending)
    leaderboard.sort(key=lambda x: x['net_worth'], reverse=True)

    # Add rank

```

```

    for i, entry in enumerate(leaderboard):
        entry['rank'] = i + 1

    socketio.emit('leaderboard_update', {'leaderboard': leaderboard}, room=room_
code)

def broadcast_game_state(room_code: str, *_args):
    """Broadcast current game state to all players in a room"""
    clock = clock_manager.get_clock(room_code)
    room = Room.get(room_code)

    if not room:
        return

    state = clock.get_state()
    # Persist clock progress so reconnects do not snap back to stale room dates.
    if room.current_date != clock.current_date:
        room.update_date(clock.current_date)
    state['start_date'] = room.start_date
    state['available_tickers'] = data_engine.get_available_tickers(clock.current
_date)
    # Use Dow before S&P 500 launch, then switch to S&P 500 from 1957 onward.
    active_index_ticker = '^GSPC' if clock.current_date >= '1957-03-01' else '^D
JI'
    state['active_index'] = {
        'ticker': active_index_ticker,
        'name': COMPANY_NAMES.get(active_index_ticker, active_index_ticker),
        'value': state['prices'].get(active_index_ticker)
    }

    socketio.emit('game_state_update', state, room=room_code)
    # Net worth depends on live prices, so refresh leaderboard on each state tic
k.
    broadcast_leaderboard(room_code)

# SocketIO event handlers
@socketio.on('join_room')
def handle_join_room(data):
    """Handle player joining a room"""
    try:
        room_code = data.get('room_code', '').upper()
        username = data.get('username', '').strip()

        if not room_code or not username:
            emit('error', {'message': 'Room code and username required'})
            return

        # Check if room exists
        if not Room.exists(room_code):
            emit('error', {'message': 'Room not found'})
            return

        # Get or create player
        player = Player.get(room_code, username)
        if not player:
            player = Player.create(room_code, username)

        # Join the socket room
        join_room(room_code)

```



```

# Store player info in session
session['room_code'] = room_code
session['username'] = player.username

# Get initial game state
clock = clock_manager.get_clock(room_code)
room = Room.get(room_code)

# Set up broadcast callback if not already set
if not clock._broadcast_callback:
    clock.set_broadcast_callback(broadcast_game_state)

if room:
    clock.set_date(room.current_date)
    clock.set_state(room.game_state)

# Send initial state to player
emit('joined_room', {
    'room_code': room_code,
    'username': player.username,
    'is_admin': player.is_admin
})

# Send current game state
broadcast_game_state(room_code)

# Send leaderboard
broadcast_leaderboard(room_code)

print(f"Player {player.username} joined room {room_code}")
except Exception as e:
    emit('error', {'message': f'Unable to join room: {str(e)}'})

```

```

@socketio.on('buy_stock')
def handle_buy_stock(data):
    """Handle buy stock request"""
    room_code = session.get('room_code')
    username = session.get('username')

    if not room_code or not username:
        emit('error', {'message': 'Not in a room'})
        return

    ticker = data.get('ticker')
    try:
        shares = float(data.get('shares', 1))
    except (TypeError, ValueError):
        emit('error', {'message': 'Invalid share amount'})
        return

    if not ticker or shares <= 0:
        emit('error', {'message': 'Invalid ticker or shares'})
        return

    # Get current price
    clock = clock_manager.get_clock(room_code)
    price = data_engine.get_price(ticker, clock.current_date)

    if price is None:
        emit('error', {'message': f'{ticker} not available yet (not IPO\'d)'})
        return

```

```

# Execute trade
player = Player.get(room_code, username)
if player.buy(ticker, shares, price):
    emit('trade_success', {
        'action': 'buy',
        'ticker': ticker,
        'shares': shares,
        'price': price,
        'total': shares * price
    })
    broadcast_leaderboard(room_code)
else:
    emit('error', {'message': 'Insufficient funds'})

@socketio.on('sell_stock')
def handle_sell_stock(data):
    """Handle sell stock request"""
    room_code = session.get('room_code')
    username = session.get('username')

    if not room_code or not username:
        emit('error', {'message': 'Not in a room'})
        return

    ticker = data.get('ticker')
    try:
        shares = float(data.get('shares', 1))
    except (TypeError, ValueError):
        emit('error', {'message': 'Invalid share amount'})
        return

    if not ticker or shares <= 0:
        emit('error', {'message': 'Invalid ticker or shares'})
        return

    # Get current price
    clock = clock_manager.get_clock(room_code)
    price = data_engine.get_price(ticker, clock.current_date)

    if price is None:
        emit('error', {'message': f'{ticker} not available'})
        return

    # Execute trade
    player = Player.get(room_code, username)
    if player.sell(ticker, shares, price):
        emit('trade_success', {
            'action': 'sell',
            'ticker': ticker,
            'shares': shares,
            'price': price,
            'total': shares * price
        })
        broadcast_leaderboard(room_code)
    else:
        emit('error', {'message': 'Insufficient shares'})

@socketio.on('disconnect')
def handle_disconnect():

```

```

"""Handle player disconnect"""
room_code = session.get('room_code')
username = session.get('username')

if room_code and username:
    leave_room(room_code)
    print(f"Player {username} left room {room_code}")

# Admin SocketIO handlers
@socketio.on('admin_toggle_play')
def handle_toggle_play(data=None):
    """Handle play/pause toggle from admin"""
    room_code = session.get('room_code')
    username = session.get('username')

    if not room_code or not username:
        return

    # Verify admin
    player = Player.get(room_code, username)
    if not player or not player.is_admin:
        emit('error', {'message': 'Admin only'})
        return

    room = Room.get(room_code)
    if not room:
        return

    # Toggle state
    clock = clock_manager.get_clock(room_code)

    # Ensure broadcast callback is set
    if not clock._broadcast_callback:
        clock.set_broadcast_callback(broadcast_game_state)

    new_state = 'playing' if room.game_state == 'paused' else 'paused'

    room.set_state(new_state)
    clock.set_state(new_state)

    broadcast_game_state(room_code)
    print(f"Room {room_code}: Game {new_state}")

@socketio.on('admin_set_date')
def handle_set_date(data=None):
    """Handle date change from admin"""
    room_code = session.get('room_code')
    username = session.get('username')

    if not room_code or not username:
        return

    # Verify admin
    player = Player.get(room_code, username)
    if not player or not player.is_admin:
        emit('error', {'message': 'Admin only'})
        return

    new_date = (data or {}).get('date')
    if not new_date:

```

```

        return

    room = Room.get(room_code)
    if not room:
        return

    # Update date
    room.update_date(new_date)
    clock = clock_manager.get_clock(room_code)
    clock.set_date(new_date)

    broadcast_game_state(room_code)
    broadcast_leaderboard(room_code)
    print(f"Room {room_code}: Date set to {new_date}")

@socketio.on('admin_reset')
def handle_reset(data=None):
    """Handle game reset from admin"""
    room_code = session.get('room_code')
    username = session.get('username')

    if not room_code or not username:
        return

    # Verify admin
    player = Player.get(room_code, username)
    if not player or not player.is_admin:
        emit('error', {'message': 'Admin only'})
        return

    room = Room.get(room_code)
    if not room:
        return

    # Reset room
    room.reset()

    # Reset all players
    players = Player.get_all_in_room(room_code)
    for p in players:
        p.reset()

    # Reset clock
    clock = clock_manager.get_clock(room_code)
    clock.reset(room.start_date)

    broadcast_game_state(room_code)
    broadcast_leaderboard(room_code)
    print(f"Room {room_code}: Game reset")

# Web routes
@app.route('/')
def index():
    """Landing page"""
    return render_template('index.html')

@app.route('/create_room', methods=['POST'])
def create_room():
    """Create a new game room"""

```

```

start_date = DEFAULT_START_DATE
username = request.form.get('username', '').strip()

if not username:
    return render_template('index.html', error='Username required')

# Generate unique room code
room_code = generate_room_code()
while Room.exists(room_code):
    room_code = generate_room_code()

# Create room
room = Room.create(room_code, start_date)

# Create admin player
player = Player.create(room_code, username, is_admin=True)

# Initialize clock
clock = clock_manager.get_clock(room_code)
clock.set_date(start_date)

return redirect(url_for('dashboard', room_code=room_code, username=username))
)

@app.route('/join_room', methods=['POST'])
def join_room_route():
    """Join an existing game room"""
    room_code = request.form.get('room_code', '').upper().strip()
    username = request.form.get('username', '').strip()

    if not room_code or not username:
        return render_template('index.html', error='Room code and username required')

    if not Room.exists(room_code):
        return render_template('index.html', error='Room not found')

    return redirect(url_for('dashboard', room_code=room_code, username=username))
)

@app.route('/dashboard')
def dashboard():
    """Game dashboard"""
    room_code = request.args.get('room_code', '').upper()
    username = request.args.get('username', '')

    if not room_code or not username:
        return redirect(url_for('index'))

    if not Room.exists(room_code):
        return redirect(url_for('index'))

    # Get room and player info
    room = Room.get(room_code)
    player = Player.get(room_code, username)

    if not player:
        player = Player.create(room_code, username)

    # Get available date range

```

```

min_date, max_date = data_engine.get_date_range()

return render_template(
    'dashboard.html',
    room_code=room_code,
    username=username,
    is_admin=player.is_admin,
    start_date=room.start_date if room else DEFAULT_START_DATE,
    min_date=min_date,
    max_date=max_date,
    tickers=TICKERS,
    company_names=COMPANY_NAMES
)

@app.route('/api/room/<room_code>')
def get_room_info(room_code):
    """Get room information"""
    room = Room.get(room_code.upper())
    if not room:
        return jsonify({'error': 'Room not found'}), 404

    clock = clock_manager.get_clock(room_code)
    prices = data_engine.get_all_prices(clock.current_date)

    return jsonify({
        'room_code': room.code,
        'start_date': room.start_date,
        'current_date': room.current_date,
        'game_state': room.game_state,
        'prices': prices
    })

@app.route('/api/player/<room_code>/<username>')
def get_player_info(room_code, username):
    """Get player information"""
    player = Player.get(room_code.upper(), username)
    if not player:
        return jsonify({'error': 'Player not found'}), 404

    clock = clock_manager.get_clock(room_code)
    prices = data_engine.get_all_prices(clock.current_date)

    return jsonify(player.to_dict(prices))

@app.route('/api/headline/<int:year>')
def get_headline(year):
    """Get one local historical headline for a specific year."""
    min_year = 1900
    max_year = 2100
    if year < min_year or year > max_year:
        return jsonify({'error': f'Year must be between {min_year} and {max_year}'
    })), 400

    available_years = sorted(HEADLINES_BY_YEAR.keys())
    selected_year = None
    for y in available_years:
        if y <= year:
            selected_year = y
    else:

```

```

        break

    if selected_year is None and available_years:
        selected_year = available_years[0]

    headline_text = HEADLINES_BY_YEAR.get(selected_year, 'Historical headline un
available.')
    data = {
        'headline': headline_text,
        'pub_date': f'{selected_year}-01-01' if selected_year else f'{year}-01-0
1',
        'url': None,
        'source': 'Historical Archive',
        'error': None
    }

    return jsonify(data)

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5001))
    socketio.run(app, host='0.0.0.0', port=port, debug=False, allow_unsafe_werkz
eug=True)
==> ./headlines_data.py <==
"""
Hardcoded historical headlines for classroom use.
At least one major event per decade from 1920 to 2020.
"""

HEADLINES_BY_YEAR = {
    1929: "Wall Street crashes, triggering the Great Depression.",
    1939: "World War II begins with Germany's invasion of Poland.",
    1945: "World War II ends after Allied victory in Europe and the Pacific.",
    1957: "Soviet Union launches Sputnik, marking the start of the Space Age.",
    1969: "Apollo 11 lands on the Moon and humans walk on its surface.",
    1973: "Oil embargo shocks global markets and accelerates inflation.",
    1989: "The Berlin Wall falls, signaling a major Cold War turning point.",
    1991: "The Soviet Union dissolves, reshaping global politics.",
    2001: "September 11 attacks transform global security policy.",
    2008: "Global financial crisis deepens after major banking failures.",
    2020: "COVID-19 pandemic disrupts economies and daily life worldwide."
}

```